

AML-3104 Neural Networks and Deep Learning

Assignment 5

Sentiment Analysis Using Recurrent Neural Networks (RNN) with
IMDB Dataset

Name: Eduardo Williams

Student Code: C0896405

Date: August 9, 2024

Contents

Understanding Sentiment Analysis and RNNs	4
What is sentiment analysis and its applications?.....	5
How RNNs differ from traditional feedforward neural networks	5
The concept of hidden states and how information is passed through time steps in RNNs	6
Common issues with RNNs such as vanishing and exploding gradients.....	6
Dataset Preparation	6
Use the IMDB dataset provided by TensorFlow for this assignment.....	6
Load the dataset and perform necessary preprocessing	7
Tokenizing the text and converting tokens to numerical format	8
Padding sequences to ensure uniform input length.....	8
Building the RNN Model	8
Implement an RNN model using TensorFlow and Keras	8
Define the model architecture	9
Compile the model with an appropriate loss function and optimizer	10
Training the Model.....	10
Split the dataset into training and validation sets.....	10
Train the model on the training set and validate it on the validation set	10
Monitor the training process and use techniques like early stopping if necessary	11
Evaluating the Model.....	12
Plot the training and validation loss and accuracy over epochs to visualize the training process.....	12
Evaluate the model's performance using appropriate metrics (e.g., accuracy, loss)	13
Analyze the results and discuss the performance of the RNN.....	13
Hyperparameter Tuning.....	13

Experiment with different hyperparameters to improve the model's performance.....	13
Document the changes and their impact on the model's performance. ..	13
Comparative Analysis	14
Implement another neural network architecture (e.g., a simple feedforward neural network) for the same task.....	14
Compare the performance of the RNN with this alternative model.....	14
Discuss the strengths and weaknesses of each approach based on your findings.....	15
References	16

Figures

Figure 1 Installing tensorflow-datasets package.....	6
Figure 2 Importing tensorflow_datasets package	7
Figure 3 Loading imdb_reviews dataset.....	7
Figure 4 RNN model	9
Figure 5 Epoch performing.....	11
Figure 6 Loss	12
Figure 7 Accuracy	12
Figure 8 ANN model.....	14
Figure 9 ANN model loss metrics	15
Figure 10 ANN model accuracy metrics	15

Tables

Table 1 Tokenizer function parameters	8
Table 2 Training parameters	10
Table 3 EarlyStopping parameters.....	11
Table 4 Metrics on test dataset evaluation.....	13

Understanding Sentiment Analysis and RNNs

Research and explain the basic concepts of sentiment analysis and Recurrent Neural Networks (RNNs). Include the following in your explanation:

What is sentiment analysis and its applications?

Sentiment analysis is a branch of NLP which evaluates opinion, emotions and feelings from text. According to Noble Desktop, sentiment analysis is a technique generally applied to online data for customer feedback assessment, especially for identifying if their comments are positive, negative or neutral.

Among diverse applications of sentiment analysis, we can name the following:

- Social media monitoring
- Brand reputation monitoring
- Political sentiment monitoring
- Customer support ticket analysis
- Brand monitoring and reputation management
- Listen to voice of the customer (VoC) and employees
- Employee satisfaction assessment
- Product analysis and improvements
- Market and competitive research

How RNNs differ from traditional feedforward neural networks

In feed-forward neural networks information flows in one direction, it means, from the input layer through hidden layers to the output layer. On the other hand, RNNs are a type of neural network that has feedback loops, it means that the information flows bi-directionally between the layers of the network.

The concept of hidden states and how information is passed through time steps in RNNs

Hidden states serve as memory and encapsulate information from previous inputs in the sequence. The output of an RNN at each time step depends not only on the current input but also on the previous hidden state. At each time step, the RNN modifies its hidden state to blend the current input with previous information, then generates an output which is carried forward to the next time step, and so on.

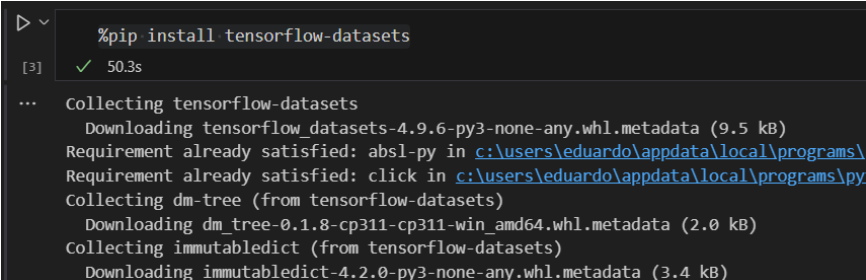
Common issues with RNNs such as vanishing and exploding gradients

Vanishing and exploding gradient problems in RNNs occur because as the gradients are propagated backwards through time, they can become very small or very large due to the repeated multiplication of gradients in each time step. Also, as an aggravating factor, recurrent connections allow to store information from previous time steps.

Dataset Preparation

Use the IMDB dataset provided by TensorFlow for this assignment

In order to use the IMDB dataset provided by Tensorflow we need to install the package and then import the required library.



```
> %pip install tensorflow-datasets
[3] ✓ 50.3s
... Collecting tensorflow-datasets
      Downloading tensorflow_datasets-4.9.6-py3-none-any.whl.metadata (9.5 kB)
      Requirement already satisfied: absl-py in c:\users\eduardo\appdata\local\programs\python\python38\lib\site-packages (from tensorflow-datasets)
      Requirement already satisfied: click in c:\users\eduardo\appdata\local\programs\python\python38\lib\site-packages (from tensorflow-datasets)
      Collecting dm-tree (from tensorflow-datasets)
      Downloading dm_tree-0.1.8-cp311-cp311-win_amd64.whl.metadata (2.0 kB)
      Collecting immutabledict (from tensorflow-datasets)
      Downloading immutabledict-4.2.0-py3-none-any.whl.metadata (3.4 kB)
```

Figure 1 Installing tensorflow-datasets package

```
▷ ▾  
# Importing necessary libraries  
import numpy as np  
from matplotlib import pyplot as plt  
  
# Importing layers and base network  
import tensorflow as tf  
import tensorflow_datasets as tfds
```

Figure 2 Importing tensorflow_datasets package

Load the dataset and perform necessary preprocessing

Upon loading the dataset, it automatically creates two prefetched Tensorflow datasets, one for training and the other one for testing. They contain 25,000 reviews each one.

```
DATASET PREPARATION  
  
Loading the IMDB dataset provided by TensorFlow  
  
# loading the dataset  
imdb = tfds.load('imdb_reviews', as_supervised=True)  
[2] ✓ 0.4s  
  
print (len(imdb['train']))  
print (len(imdb['test']))  
[3] ✓ 0.0s  
... 25000  
    25000
```

Figure 3 Loading imdb_reviews dataset

Tokenizing the text and converting tokens to numerical format

For tokenization, we used the **Tokenizer** function from **tensorflow.keras.preprocessing.text** package. Upon defining the tokenizer object some parameters were required:

Parameter	Description
num_words	Size of entire vocabulary
filters	Regex pattern to exclude symbols and other characters from strings
lower	To convert reviews to lowercase
split	Separator of words
oov_token	Token for unknown words

Table 1 Tokenizer function parameters

In addition, we used **texts_to_sequences** method to convert reviews from string to numerical vectors.

Padding sequences to ensure uniform input length

Since not all reviews have the same number of words, we needed to fix them to the same length. For this, we used **pad_sequences** method, which receives as input the previous sequences vector and the maximum number of words.

Building the RNN Model

Implement an RNN model using TensorFlow and Keras

Implement an RNN model using TensorFlow and Keras

```
# Define the model architecture

# Input layer
# Input for variable-length sequences of integers
inputs = Input(shape=(None,), dtype="int32")

# Embedding layer
# Embed each integer
x = Embedding(input_dim = vocab_size,
              output_dim = embed_size,
              input_length = max_len)(inputs)

# RNN layer (use LSTM or GRU for better performance)
# Add 2 bidirectional LSTMs
x = Bidirectional(LSTM(64, return_sequences=True))(x)
x = Bidirectional(LSTM(64))(x)

# Output layer
# Add a classifier
outputs = Dense(1, activation="sigmoid")(x)

# tf.keras.Model
rnn_model = Model(inputs, outputs)
```

8] ✓ 0.2s

Figure 4 RNN model

Define the model architecture

We defined an RNN model using TensorFlow and Keras. First of all, we defined a variable vector shape for the input layer. However, later we defined an Embedding layer, which takes the same length as the padded sequences for the **input_length** parameter. In addition, we set the vocabulary size to the **input_dim** parameter.

We also defined two LSTM layers of 64 units, and finally we defined the output layer with a Sigmoid activation function.

Compile the model with an appropriate loss function and optimizer

For compiling the model, we used the following parameters:

- **loss:** since the prediction can be take just two values (positive or negative) then we set it as **binary_crossentropy**
- **optimizer:** it was setting as **adam**, which is equivalent to mini-batch gradient descent
- **metrics:** we included **accuracy** as a metric

Training the Model

Split the dataset into training and validation sets

When we loaded the dataset, it automatically created two prefetched Tensorflow datasets, one for training and the other one for testing. They contain 25,000 reviews each one.

Train the model on the training set and validate it on the validation set

For training process, we took 20% from training dataset for validation, furthermore, we defined the following parameters:

Parameter	Value
validation_split	0.2
epochs	20
batch_size	32
verbose	1

Table 2 Training parameters

Monitor the training process and use techniques like early stopping if necessary

Additionally, we used one callback: EarlyStopping, for monitoring when accuracy no longer improves to stop the fitting process. Worth mentioning that we enabled **restore_best_weights** parameter.

Parameter	Value
monitor	val_loss
mode	min
patience	5
min_delta	0.01
verbose	1
restore_best_weights	True

Table 3 EarlyStopping parameters

Following picture shows the training performing.

```
Epoch 1/20
625/625 ————— 125s 194ms/step - accuracy: 0.6939 - loss: 0.5635 - val_accuracy: 0.8312 - val_loss: 0.4027
Epoch 2/20
625/625 ————— 128s 205ms/step - accuracy: 0.8773 - loss: 0.3088 - val_accuracy: 0.7430 - val_loss: 0.5612
Epoch 3/20
625/625 ————— 131s 210ms/step - accuracy: 0.9046 - loss: 0.2549 - val_accuracy: 0.8308 - val_loss: 0.4069
Epoch 4/20
625/625 ————— 119s 190ms/step - accuracy: 0.9491 - loss: 0.1428 - val_accuracy: 0.8492 - val_loss: 0.4421
Epoch 5/20
625/625 ————— 125s 199ms/step - accuracy: 0.9717 - loss: 0.0900 - val_accuracy: 0.8376 - val_loss: 0.4954
Epoch 6/20
625/625 ————— 126s 201ms/step - accuracy: 0.9827 - loss: 0.0587 - val_accuracy: 0.8434 - val_loss: 0.4893
Epoch 6: early stopping
```

Figure 5 Epoch performing

Evaluating the Model

Plot the training and validation loss and accuracy over epochs to visualize the training process

Followings pictures show the results both for loss and accuracy in training (green line) and validation (orange line) datasets:

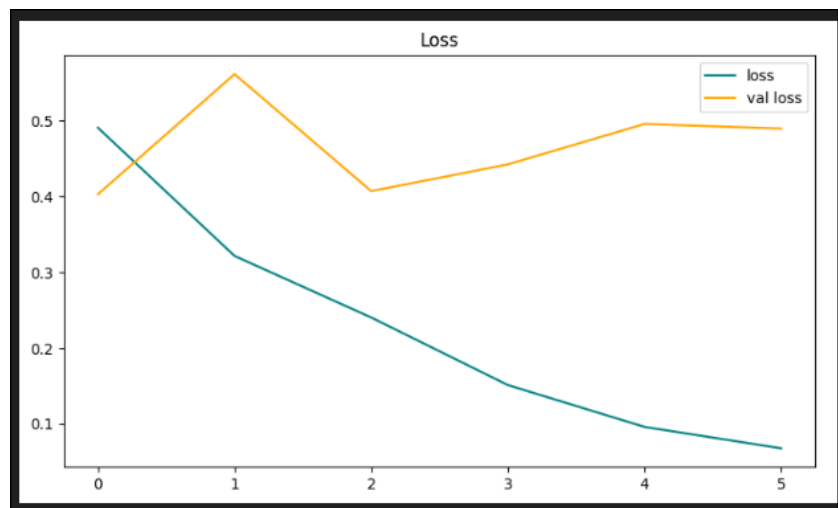


Figure 6 Loss

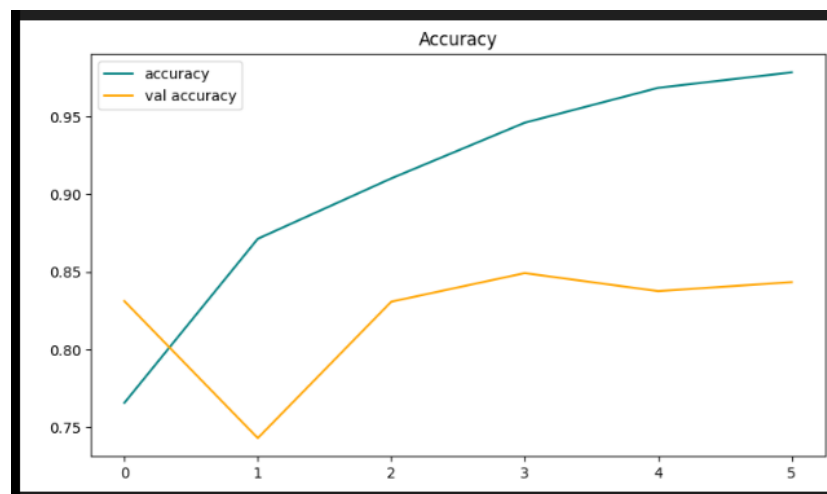


Figure 7 Accuracy

Evaluate the model's performance using appropriate metrics (e.g., accuracy, loss)

After training, we evaluated the model's performance on test dataset. We got the results shown in the following table:

Metric	Value
Accuracy	84%
Loss	37%

Table 4 Metrics on test dataset evaluation

Analyze the results and discuss the performance of the RNN

So far, we got good results on train dataset, however on test dataset the accuracy decreased.

Hyperparameter Tuning

Experiment with different hyperparameters to improve the model's performance.

Document the changes and their impact on the model's performance.

In this part we tried to run a grid search cross validation, however it took a lot of time and did not return good results.

Comparative Analysis

Implement another neural network architecture (e.g., a simple feedforward neural network) for the same task

Finally, we designed a simple ANN in order to compare its results against previous ones.

```
# building an ANN model
model_ffn = Sequential()
model_ffn.add(Embedding(input_dim = vocab_size, output_dim = embed_size))
model_ffn.add(Flatten())
model_ffn.add(Dense(128, activation = 'relu'))
model_ffn.add(Dropout(0.3))
model_ffn.add(Dense(64, activation = 'relu'))
model_ffn.add(Dropout(0.3))
model_ffn.add(Dense(1, activation='sigmoid'))
✓ 0.0s
```

Figure 8 ANN model

Compare the performance of the RNN with this alternative model

As we can see in the following pictures, with this model metrics tend to be stable instead of increasing (accuracy) or decreasing (loss).

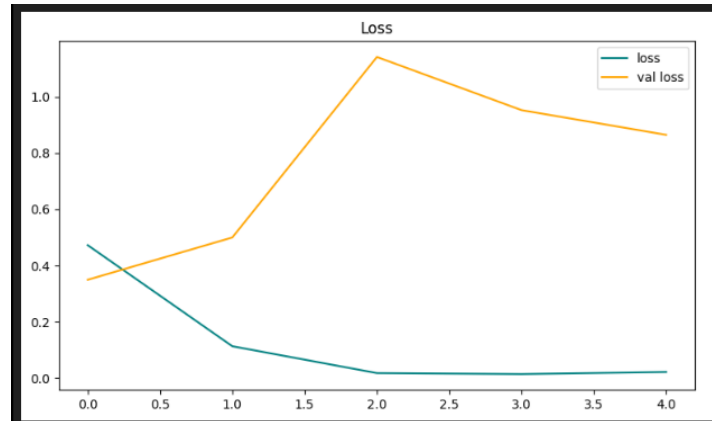


Figure 9 ANN model loss metrics

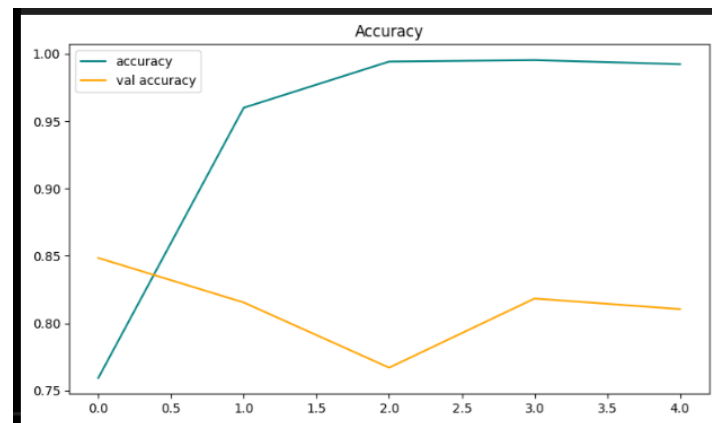


Figure 10 ANN model accuracy metrics

Discuss the strengths and weaknesses of each approach based on your findings

ANN is faster than RNN model, however RNN gives better results. RNN takes a lot of time in training process, but it offers more possibilities for adding layers and configuring it.

References

Mediun. (n.d.). *Feed-forward and Recurrent Neural Networks: The Future of Machine Learning*. Retrieved from [https://medium.com:
https://medium.com/@prajeeshprathap/feed-forward-and-recurrent-
neural-networks-the-future-of-machine-learning-8b2c1975f0c5](https://medium.com/@prajeeshprathap/feed-forward-and-recurrent-neural-networks-the-future-of-machine-learning-8b2c1975f0c5)

Noble desktop. (n.d.). *Real-World Applications of Sentiment Analysis*. Retrieved from noble desktop: [https://www.nobledesktop.com/classes-
near-me/blog/sentiment-analysis-real-world-
applications#:~:text=The%20following%20are%20some%20of,%2C%2
0specific%20products%2C%20or%20features.](https://www.nobledesktop.com/classes-near-me/blog/sentiment-analysis-real-world-applications#:~:text=The%20following%20are%20some%20of,%2C%20specific%20products%2C%20or%20features.)